Mysql

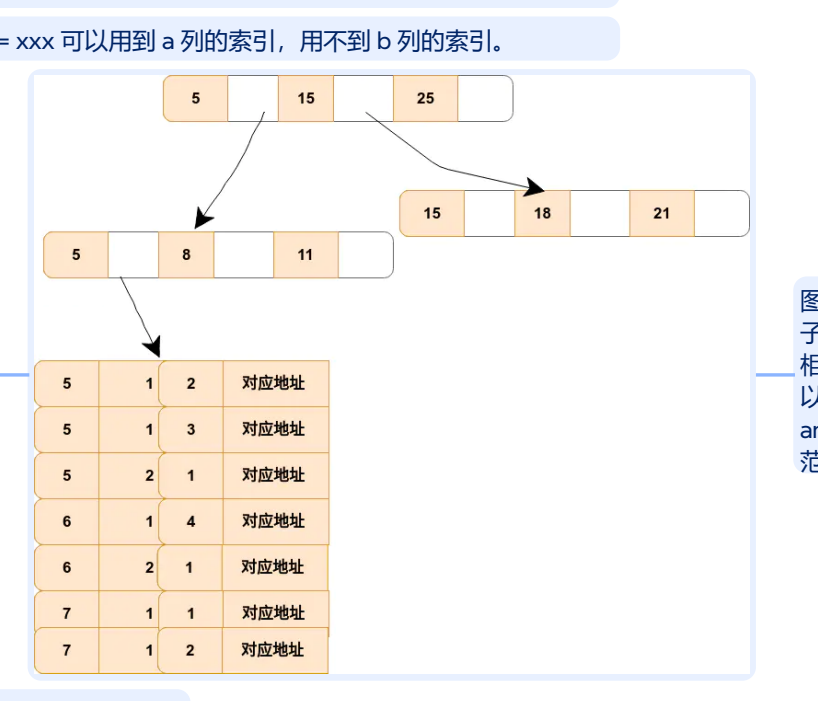
InnoDB 逻辑存储结构

表设计

- 通俗地理解三个范式

索引

- 索引数据结构
  - B+树的特点 — B+树索引是一个m叉树
    - 1/每个节点中子节点的个数不能超过 m，也不能小于 m/2；
    - 2/根节点的子节点个数可以不超过 m/2，这是一个例外；
    - 3/m 叉树只存储索引，并不真正存储数据，这个有点儿类似跳表；
    - 4/通过链表将叶子节点串联在一起，这样可以方便的按区间查找；
    - 5/根节点会被存储在内存中，其他节点存储在磁盘中。
  - B树 — 一个m阶的B-Tree有以下性质
    1. **每个节点最多有m个子节点；**
    2. **每个非叶子节点（根节点除外）至少含有m/2个子节点；**
    3. 如果根节点不是叶子节点，那么根节点至少有两个子节点；
    4. **每个节点上，所有的关键字都是有序的，从左到右，依次从小到大排序；**
    5. 每个关键字的左子树的均值小于当前关键字，右子树的均值大于当前关键字；
    6. **每个节点都存有索引和数据；**
    7. 对于一个非叶子节点而言，它最多能存储m-1个关键字；
    8. 所有叶子节点位于同一层。
  - B*树(Oracle)
    - B+树B*的不同之处
      - 首先是关键字个数限制问题，B+树初始化的关键字初始化个数是cei(m/2)，b*树的初始化个数为（cei(2/3*m)）
      - B+树节点满时就会分裂，而B*树节点满时会检查兄弟节点是否满（因为每个节点都有指向兄弟的指针），如果兄弟节点未满则向兄弟节点转移关键字，如果兄弟节点已满，则从当前节点和兄弟节点各拿出1/3的数据创建一个新的节点出来
    - B*树的一些关键特性
      - 节点填充策略
        - B树在节点分裂时采用不同的策略。在传统的B树中，当一个节点满时，它会被分裂成两个节点，每个节点包含一半的键。
        - B*树要求每个节点至少填充到2/3的容量。这意味着，增加了节点的利用率。
      - 减少树的高度 — 由于B*树的节点利用率更高，这通常导致树的高度比标准B树更低。树的高度减少意味着在大多数情况下可以减少磁盘I/O操作
      - 查询性能的提高 — 由于树的高度较低，B*树在执行查找操作时通常比B树更快。
      - 写入性能 — 虽然B*树的写入操作（如插入和删除）可能需要更复杂的节点重平衡策略，但总体上，由于树的高度较低，其写入性能通常优于或等同于标准B树。
      - 空间效率 — B*树通过保持节点至少为2/3满，提高了空间利用率。
      - 应用于索引
    - TODO
  - B+和B树对比
    - **1/B+树中的节点不存储数据，只是索引，而B树中的节点存储数据.**
    - **2/B树中的叶子节点并不需要链表来串联，B+树更容易的进行区间遍历甚至全部遍历。**
    - 3/也就是说，B树只是一个每个节点的子节点个数不小于m/2的m叉树。
- 索引存储 — 页
  - 在 MySQL 中数据读取的基本单位都是页，所以我们这里叫做页更符合 MySQL 中索引的底层数据结构。
  - 数据库中页的大小是固定的，InnoDB 中页的默认大小是 16KB
- 聚集索引 — 一般情况下聚簇索引等于主键索引，但聚簇索引又不完全等于主键索引，因为一张表中没有主键索引，那么聚簇索引会使用第一个唯一索引（此列必须为 not null），如果以上情况都不满足，那么 InnoDB 会生成一个隐藏的聚簇索引
- 非聚集索引
  - 定义 — 辅助索引，Secondary Index，也称为非聚集索引 — 以主键以外的列值作为键值构建的 B+ 树索引，我们称之为非聚集索引
  - 区别
    - 非聚集索引的叶子节点不存储表中的数据，而是存储该列对应的主键，想要查找数据我们还需要根据主键再去聚集索引中进行查找，这个再根据聚集索引查找数据的过程，我们称为回表
    - 辅助索引的存在并不影响数据在聚集索引中的组织，因此每张表上可以有多个辅助索引，但只能有一个聚集索引
    - 聚集索引的叶节点就是数据节点。而非聚簇索引的叶节点仍然是索引节点，只不过有一个指针指向对应的数据块
  - 非聚集索引又分唯一非聚集索引和非唯一两种
- 联合索引
  - 最左匹配原则 — 在通过联合索引检索数据时，从索引中最左边的列开始，一直向右匹配，如果遇到范围查询(>、<、between、like等)，就停止后边的匹配。使用函数、运算表达式及类型隐式转换等，也无法使用索引。
  - 例 — 对字段 (a, b, c) 建立联合索引
    - 用到联合索引的一部分
      - where a = xxx and c = xxx 可以用到 a 列的索引，用不到 c 列索引。
      - where a like 'xxx%' and b = xxx 可以用到 a 列的索引，用不到 b 列的索引。
      - where a > xxx and b = xxx 可以用到 a 列的索引，用不到 b 列的索引。
    - 构建一颗 B+ 树只能根据一个值来构建，因此数据库依据联合索引最左的字段来构建 B+ 树



图就是一个形如(a,b,c)联合索引的 b+ 树，其中的非叶子节点存储的是第一个关键字的索引 a，而叶子节点存储的是三个关键字的数据。这里可以看出 a 是有序的，而 b，c 都是无序的。但是当在 a 相同的时候，b 是有序的，b 相同的时候，c 又是有序的。通过对联合索引的结构的了解，那么就可以很好的了解为什么最左匹配原则中如果遇到范围查询就会停止了。以 select * from t where a=5 and b>0 and c =1; #这样a,b可以用到（a,b,c），c不可以 为例子，当查询到 b 的值以后（这是一个范围值），c 是无序的。所以就不能根据联合索引来确定到底该取哪一行。

- 索引覆盖
  - 建立了联合索引后，直接在索引中就可以得到查询结果，从而不需要回表查询聚簇索引中的行数据信息
  - 好处
    - 辅助索引不包含行数据的所有信息，故其大小远小于聚簇索引，因此可以减少大量的IO操作。
    - 索引覆盖只需要扫描一次索引树，不需要回表扫描聚簇索引树，所以性能比回表查询要高。
    - 索引中列值是按顺序存储的，索引覆盖能避免范围查询回表带来的大量随机IO操作。
- 索引下推
  - 索引下推是把本应该在 server 层进行筛选的条件，下推到存储引擎层来进行筛选判断，这样能有效减少回表
  - 例 — 联合索引（name，age）
    - select * from t_user where name like 'L%' and age = 17; — 只有name用的索引，但是age并没有用到
    - 不用索引下推的执行过程
      - 第一步：利用索引找出name带'L'的数据行：LiLei、Lili、Lisa、Lucy 这四条索引数据
      - 第二步：再根据这四条索引数据中的 id 值，逐一进行回表扫描，从聚簇索引中找到相应的行数据，将找到的行数据返回给 server 层。
      - 第三步：在server层判断age = 17,进行筛选，最终只留下 Lucy 用户的数据信息。
    - 使用索引下推的执行过程
      - 第一步：利用索引找出name带'L'的数据行：LiLei、Lili、Lisa、Lucy 这四条索引数据
      - 第二步：根据 age = 17 这个条件，对四条索引数据进行判断筛选，最终只留下 Lucy 用户的数据信息。（注意：这一步不是直接进行回表操作，而是根据 age = 17 这个条件，对四条索引数据进行判断筛选）
      - 第三步：将符合条件的索引对应的 id 进行回表扫描，最终将找到的行数据返回给 server 层。
    - 比较二者的第二步我们发现，索引下推的方式极大的减少了回表次数

事务

- ACID
  - 1)原子性(Atomic)：事务中各项操作，要么全做要么全不做，任何一项操作的失败都会导致整个事务的失败；
  - 2)一致性(Consistent)：事务结束后系统状态是一致的；
  - 3)隔离性(Isolated)：并发执行的事务彼此无法看到对方的中间状态；
  - 4)持久性(Durable)：事务完成后所做的改动都会被持久化，即使发生灾难性的失败。通过日志和同步备份可以在故障发生后重建数据。
- 事务隔离级别
  - 分类
    - Read UnCommitted（读未提交）
      - 是什么 — 最低的事务隔离级别，一个事务还没提交时，它做的变更就能被别的事务看到
      - 存在问题 — 脏读、不可重复读、幻读
    - Read Commited（读已提交）
      - 是什么 — 保证一个事物提交后才能被另外一个事务读取
      - 大多数数据库的默认级别就是 Read committed，比如 Sql Server , Oracle
      - 存在问题 — 不可重复读、幻读
    - Repeatable Read（可重复读）
      - 是什么 — 多次读取同一范围的数据会返回第一次查询的快照，即使其他事务对该数据做了更新修改。事务在执行期间看到的数据前后必须是一致的
      - 原理 — 在MySQL的InnoDB存储引擎中，可重复读是默认的事务隔离级别，它通过使用行级锁和一致性读（即读取数据的快照版本）来实现这一隔离级别的语义
      - 存在问题 — 幻读
    - Serializable（串行化）
      - 是什么 — "写"会加"写锁"，"读"会加"读锁"。当出现读写锁冲突的时候，后访问的事务必须等前一个事务执行完成，才能继续执行
      - 不存在问题 — 事务 100% 隔离，可避免脏读、不可重复读、幻读的发生
  - 问题
    - 脏读 — 定义 — 就是一个事务在处理过程中读取了另外一个事务未提交的数据
    - 不可重复读（non-repeatable read）
      - 定义 — 不可重复读的问题发生在**一个事务中两次读取**同一记录时，发现它的数据在两次读取之间被另一个事务修改过了
      - 原因 — 由于其他事务对这些数据行进行了更新
      - 例 — 如果事务T1读取了数据行R，然后事务T2更新了行R，并且T1再次读取行R，此时T1将看到不同的数据
      - 例

时间点	事务A	事务B
1	开启事务A	
2		开启事务B
3	查询余额为100	
4		余额增加至150
5	查询余额为100	
6		提交事务
7	查询余额为150	

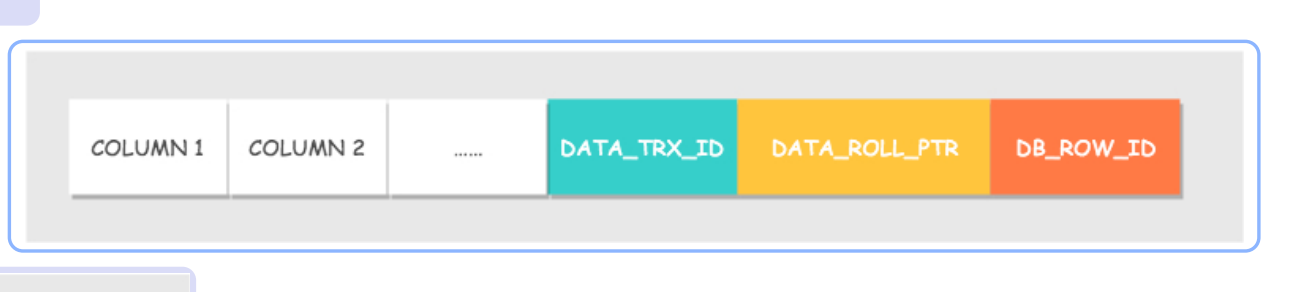
      - 和脏读区别 — 脏读是某一事务读取了另外一个事务未提交的数据，不可重复读是读取了其他事务提交的数据
    - 幻读(phantom read)
      - 定义 — 幻读是指当一个事务在对数据库中某个**范围的记录**进行操作时（如查询），**另一个事务又在该范围内插入或删除了一些记录**，导致第一个事务在再次查询该范围的记录时，会发现一些原本不存在的"幻影"记录 — The so-called phantom problem occurs within a transaction when the same query produces different sets of rows at different times. For example, if a SELECT is executed twice, but returns a row the second time that was not returned the first time, the row is a "phantom" row.
      - 原因 — 这种问题通常是由于其他事务在查询间隙插入或删除了数据行，导致查询结果的行数发生变化。
      - 例 — 如果事务T1读取了满足某条件的所有行，然后事务T2插入了一些新行，这些新行也满足T1的查询条件。如果T1再次执行相同的查询，它将看到之前未出现的新行。
      - 例 — repeatable read下，如果有2个线程，一个线程T1对一条数据修改，读到100+1，另一个线程T2读数据读到的是多少? 1、如果线程T1未提交，线程2读到的是多少? 2、如果线程T1已提交，线程2读到的是多少? — **T2读到的数据，依赖，生成的读视图(Read View)的时间。① RC下，执行每条sql时，生成的读视图(Read View)。②RR情况下，事务开始后，第一次select时生成的读视图(Read View)，同时要考虑T2是否是当前读**
        - 如果T2在T1提交修改前开始读取数据 — 在可重复读隔离级别下，T2在整个事务期间都将看到这个初始的快照数据，即使T1在此期间提交了修改。
        - 如果T2在T1提交修改后开始读取数据 — T2将读到修改后的数据，即101。因为在可重复读隔离级别下，一旦T1提交了修改，这些修改对其他事务就变得可见
      - 两点说明
        - **在可重复读隔离级别下，普通查询是快照读，是不会看到别的事务插入的数据的，幻读只在"当前读"下才会出现**
        - 幻读专指"新插入的行"，读到原本存在行的更新结果不算。因为"当前读"的作用就是能读到所有已经提交记录的最新值
      - 例

时间点	事务A	事务B
1	开启事务A	
2		开启事务B
3	查询id<3的所有记录，共3条	
4		插入一条记录id=2
5		提交事务
6	查询id<3的所有记录，共4条	

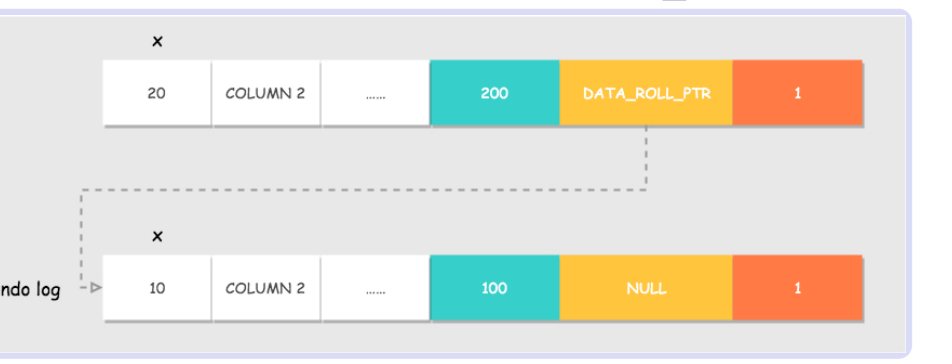
    - 不可重复读和幻读区别
      - 数据变化的类型 — 不可重复读关注的是单个或多个已存在数据行的内容变化（更新），而幻读关注的是因为新行的插入或删除导致的整体结果集的变化
      - 解决办法 — 在SQL标准的事务隔离级别中，不可重复读一般通过"可重复读"（Repeatable Read）隔离级别解决，而幻读则需要更高级别的"可串行化"（Serializable）隔离级别来解决。
      - 本质区别 — 可重复读中，该sql第一次读取到数据后，就将这些数据加锁，其它事务无法修改这些数据，就可以实现可重复读了 — 但这种方法却无法锁住insert的数据 — 当事务A先前读取了数据，或者修改了全部数据，事务B还是可以insert数据提交，这时事务A就会发现莫名其妙多了一条之前没有的数据，这就是幻读，不能通过行锁来避免
  - 解决方案
    - 图

隔离级别	脏读	不可重复读	幻读
read uncommitted（未提交读）	√	√	√
read committed（提交读）	×	√	√
repeatable read（可重复读）	×	×	√
serialization（可串行化）	×	×	×

      - Mysql默认的事务隔离级别是可重复读(Repeatable Read)
      - Oracle的默认隔离级别就是Read Commited
    - 总结
      - 读未提交：别人改数据的事务尚未提交，我在我的事务中也能读到。
      - 读已提交：别人改数据的事务已经提交，我在我的事务中才能读到。
      - 可重复读：别人改数据的事务已经提交，我在我的事务中也不去读。
      - 串行：我的事务尚未提交，别人就别想改数据。
- 隔离级别的实现
  - 视图（read-view）
    - 事务的机制是通过视图（read-view）来实现的并发版本控制（MVCC），不同的事务隔离级别创建读视图的时间点不同。
      - **- 读未提交是不创建，直接返回记录上的最新值**
      - **- 读已提交是每条 SQL 创建读视图，在每个 SQL 语句开始执行的时候创建的。隔离作用域仅限该条 SQL 语句。**
      - **- 可重复读是每个事务重建读视图，整个事务存在期间都用这个视图。**
      - **- 串行化隔离级别下直接用加锁的方式来避免并行访问。**
    - 是什么 — 视图可以理解为数据副本/快照，每次创建视图时，将当前"已持久化的数据"创建副本，后续直接从副本读取，从而达到数据隔离效果 — 数据表其实还有一些**隐藏的属性**，比如每一行的事务 id，所以**每一行数据可能会有多个版本**，每一个修改过它的事务都会有一行，并且还会有**关联的 undo 日志**，表示这个操作原来的数据是什么，可以用它做回滚。
    - 作用 — 用来做可见性判断
    - ReadView属性（这些属性共同工作，确保了即使在并发环境中，事务也能保持其隔离级别，实现了非锁定读取，如在REPEATABLE READ和READ COMMITTED隔离级别下的SELECT操作）
      - **m_ids：当前系统中活跃的事务ID列表。这些是在生成ReadView时正在执行且尚未提交的事务**
      - **min_trx_id：生成ReadView时活跃事务中的最小事务ID。它代表了可见更改的最小版本**
      - **max_trx_id：生成ReadView时系统中下一个将要分配的事务ID。它代表了在当前事务开始之后进行的所有更改都不应该是可见的**
      - **creator_trx_id：创建ReadView的事务的ID**
      - up_limit_id：已经提交并且对于创建ReadView的事务可见的最大事务ID
      - low_limit_id：对于创建ReadView的事务不可见的最小事务ID
      - set 或 trx_ids：一个事务ID的集合，包含了那些在ReadView创建时已经开始但还没有提交的事务的ID。这有助于确定哪些更改是对当前事务可见的
    - ReadView可见性判断
      - 规则
        - 事务ID小于ReadView的min_trx_id — 如果数据行的版本是由事务ID小于min_trx_id的事务创建的，那么这个版本对当前事务是可见的。因为min_trx_id之前的所有事务都已经提交，其更改对所有新的事务都是可见的
        - 事务ID**在ReadView的m_ids列表中** — 如果数据行的版本是由还未提交且在m_ids列表中的事务创建的，那么这个版本对当前事务是不可见的。因为这些事务可能在当前事务之后提交，它们的更改在当前事务的快照中不应该出现
        - 事务ID大于或等于ReadView的max_trx_id — 如果数据行的版本是由事务ID大于或等于max_trx_id的事务创建的，那么这个版本对当前事务是不可见的。这代表了在ReadView创建之后开始的事务，其更改不应对当前事务可见
        - 其他情况
          - 如果数据行的版本是由事务ID大于min_trx_id且**不在m_ids列表**中的事务创建的，且事务ID小于max_trx_id，则这个版本对当前事务是可见的。这意味着这个版本是由在当前事务之前提交的事务创建的
          - 如果被访问版本的 transaction_id 属性值与 ReadView 中的 **creator_trx_id 值相同**，表明当前事务在访问它自己修改过的记录，所以该版本可以被当前事务访问。
      - 特别情况处理
        - 删除标记的行 — 如果一个数据行被一个事务标记为删除，这个行的版本只有在创建该行的事务对当前事务可见时才可见。如果删除操作本身不可见，则整个行都不可见
        - READ COMMITTED隔离级别 — 在这个隔离级别下，每个新的SELECT操作都会创建一个新的ReadView，因此事务中后续的查询可能看到其他事务提交的新更改
    - ReadView生成时机
      - RC — **每次读取数据前，都生成一个 ReadView**
      - RR
        - **在当前事务第一次读取数据时，生成一个 ReadView，之后的查询操作都重复使用这个 ReadView**
        - 注意：开启事务后第一个select语句才是快照读的地方，而不是一开启事务就快照读
  - undo log
    - undo log 中存储的是老版本数据，使用 undo 日志来实现回滚操作，保证事务的一致性。
    - undo log主要分为两种
      - insert undo log — 事务对 insert 新记录时产生的 undolog，只在事务回滚时需要，并且在事务提交后就可以立即丢弃。
      - update undo log — 事务对记录进行 delete 和 update 操作时产生的 undo log。不仅在事务回滚时需要，一致性读也需要，所以不能随便删除，只有当数据库所使用的快照中不涉及该日志记录，对应的回滚日志才会被 purge 线程删除。
  - 一个事务读取某一行的数据到底返回什么结果呢? — 取决于隔离级别
    - 如果是 Read Committed，那么返回的是**最新的事务的提交值**，所以未提交的事务修改的值是不会读到的，这就是 Read Committed 实现的原理。
    - 如果是 Read Repeatable 级别，那么只能返回发起时间比当前事务早的事务的提交值，和比当前事务晚的删除事务删除的值。这其实就是 MVCC 方式
- 事务的实现
  - 在InnoDB中的每一条记录实际都会存在三个隐藏列
    - DB_TRX_ID — 事务 ID，是根据事务产生时间顺序自动递增的，是独一无二的
    - DB_ROLL_PTR — 回滚指针，本质上就是一个指向记录对应的undo log的一个指针，InnoDB 通过这个指针找到之前版本的数据
    - DB_ROW_ID — 主键，如果有自定义主键，那么该值就是主键；如果没有主键，那么就会使用定义的第一个唯一索引；如果没有唯一索引，那么就会默认生成一个隐藏列作为主键。



  - Undo Log 链 — 在多个事务并行操作某行数据的情况下，不同事务对该行数据的 UPDATE 会产生多个版本，然后通过回滚指针组织成一条 Undo Log 链.



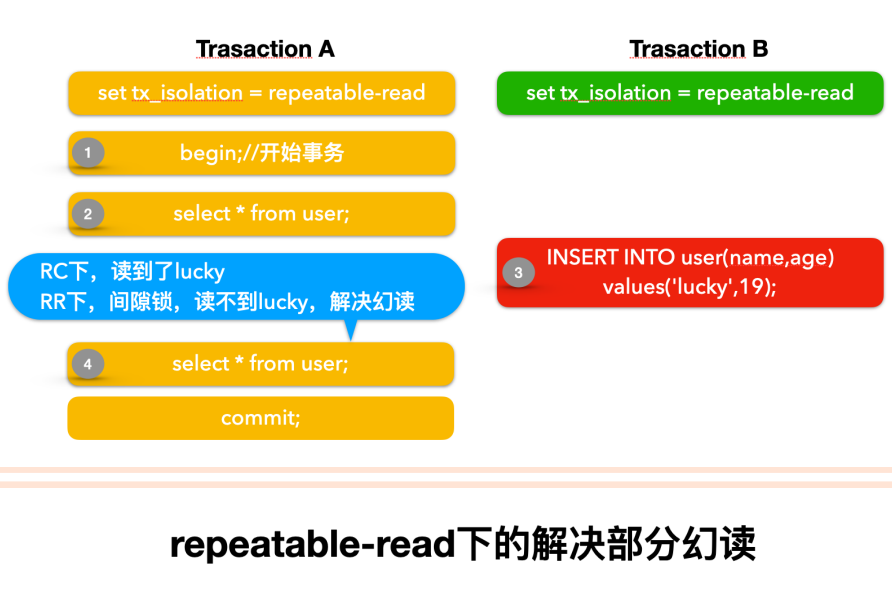
- XA
  - 是什么
    - XA 规范 是 X/Open 组织定义的分布式事务处理（DTP，Distributed Transaction Processing）标准
    - XA 规范 描述了全局的事务管理器与局部的资源管理器之间的接口。XA规范的目的是允许的多个资源（如数据库，应用服务器，消息队列等）在同一事务中访问，这样可以使 ACID 属性跨越应用程序而保持有效。
    - 使用两阶段提交（2PC，Two-Phase Commit）来保证所有资源同时提交或回滚任何特定的事务
  - 历史 — XA 规范 在上世纪 90 年代初就被提出。目前，几乎所有主流的数据库都对 XA 规范 提供了支持。

MVCC

- 是什么 — 维持一个数据的多个版本，使得读写操作没有冲突
- 原理
  - 通过数据行的历史版本来实现数据库的并发控制
  - 实现主要是通过 版本链 + ReadView 结构完成
- 思想 — 保存数据的历史版本。这样一个事务进行查询操作时，就可以通过比较版本号来判断哪个较新的版本对当前事务可见。
- MVCC 是为了解决什么问题
  - 锁机制可以控制并发操作,但是其系统开销较大,而 MVCC 可以在大多数情况下代替行级锁,使用 MVCC,能降低其系统开销.
  - **RR下，对于快照读来说，幻读的解决是依赖mvcc解决。而对于当前读则依赖于gap-lock解决**
- MVCC涉及到的名词
  - 快照读（普通读） — 像'不加锁'的select操作就是快照读，即不加锁的非阻塞读
  - 当前读
    - 是什么
      - 读取的是记录的最新版本，读取时还要保证其他并发事务不能修改当前记录，会对读取的记录进行加锁
      - 更新数据都是先读后写的，而这个读，只能读当前的值，称为"当前读"（current read）
    - 例 — select lock in share mode(共享锁), select for update(排他锁); update(排他锁), insert(排他锁), delete(排他锁)
    - 作用 — 保证了当前事务修改数据时，不会丢失其他事务已经提交的修改
  - Read View — 见上一节
  - 行锁 Record Lock
    - 解决什么问题 — 在并发环境下，多个事务对同一数据行进行修改时可能引发的数据不一致性问题。行锁通过为数据行加锁，确保在任何给定时刻，只有一个事务可以修改该行，**不同的事务也可以同时修改不同的数据行，从而显著提高了数据库的并发处理能力**
    - 存在范围 — 所有的隔离级别下被使用
  - 间隙锁 Gap Lock
    - 作用
      - 间隙锁主要用来防止幻读
      - 用在repeatable-read隔离级别下，指的是当对数据进行条件，范围检索时，对其范围内也许并存在的值进行加锁! — 当查询的索引含有唯一属性（唯一索引，主键索引）时，Innodb存储引擎会对next-key lock进行优化，将其降为record lock,即仅锁住索引本身，而不是范围！若是普通辅助索引，则会使用传统的next-key lock进行范围锁定!
    - 作用范围
      - 1. 对主键或唯一索引，如果当前读时，where条件全部精确命中(=或者in)，Innodb存储引擎会对next-key lock进行优化，将其降为record lock（记录锁），即仅锁住索引本身，而不是范围。
      - 2. 非唯一索引列，如果where条件部分命中(>、<、like等)或者全未命中，则会加附近Gap间隙锁。例如，某表数据如下，非唯一索引2,6,9,9,11,15。如下语句要操作非唯一索引列9的数据，gap锁将会锁定的列是(6,11]，该区间内无法插入数据。如果是对条件是2或15，是不是锁全表?
      - 3. 没有索引的列，当前读操作时，会加全表gap间隙锁，生产环境要注意。
    - 存在范围 — 只有在 Read Repeatable 、Serializable 隔离级别才有
  - 临键锁 Next-key — Record Lock + Gap Lock，锁定一个范围并且锁定记录本身
- MVCC适用场景
  - 1. MVCC 手段只适用于 Msyql 隔离级别中的读已提交（Read committed）和可重复读（Repeatable Read）
  - 2. 串行化由于是会对所涉及到的表加锁，并非行锁，自然也就不存在行的版本控制问题

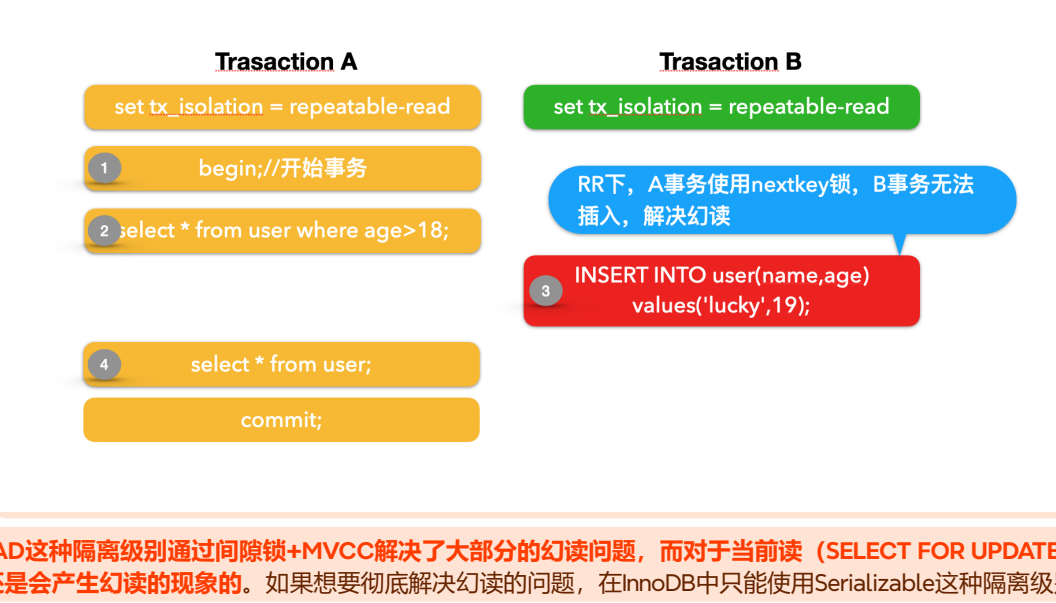
RR下MVCC与幻读

- 间隙锁解决幻读的情况
  - ① RC下，执行每条sql时，生成的读视图(Read View) — 事务中每一条 SELECT 语句均可以看到其他已提交事务对数据的修改，那么只要该事物已经提交其结果就是可见的。读操作需要加共享锁，但是在语句执行完以后释放共享锁
  - ②RR情况下，事务开始后，第一次select时生成的读视图(Read View)
    - RR 的本质是从第一个操作语句生成 ReadView 开始，任何已经提交过的事务的修改均可见。读操作需要加共享锁，但是在**事务提交之前并不释放共享锁**，也就是必须等待事务执行完毕以后才释放共享锁
    - 例



repeatable-read下的解决部分幻读

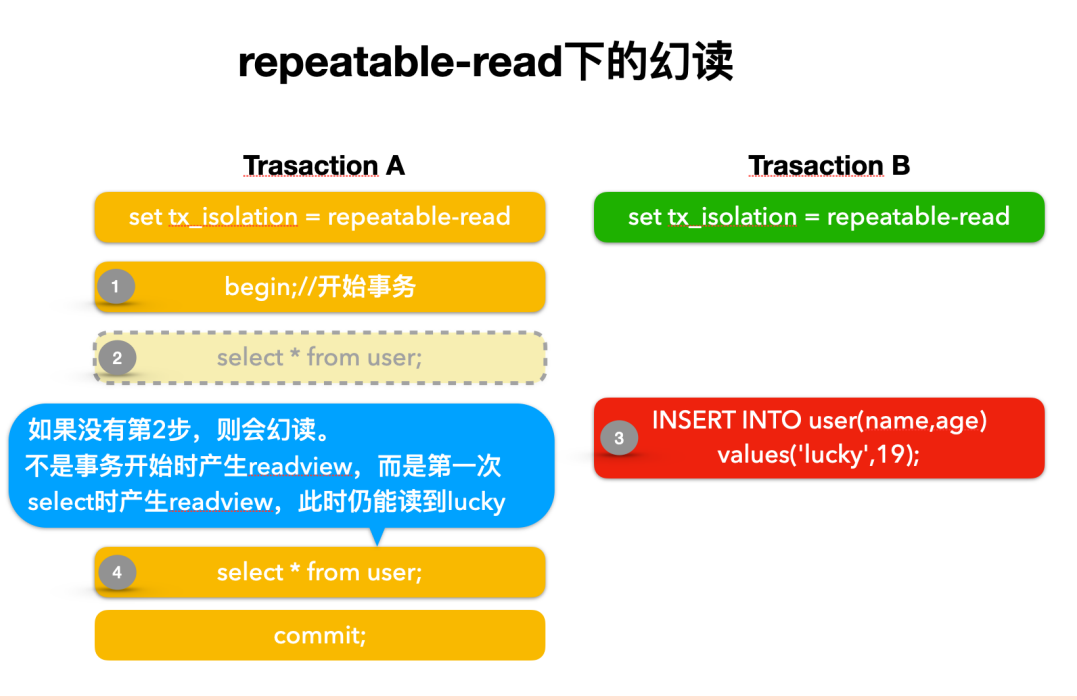
Trasaction A
Trasaction B
set tx_isolation = repeatable-read
set tx_isolation = repeatable-read
1 begin;//开始事务
2 select * from user;
3 INSERT INTO user(name,age) values('lucky',19);
RC下，读到了lucky
RR下，间隙锁，读不到lucky，解决幻读
4 select * from user;
commit;



repeatable-read下的解决部分幻读

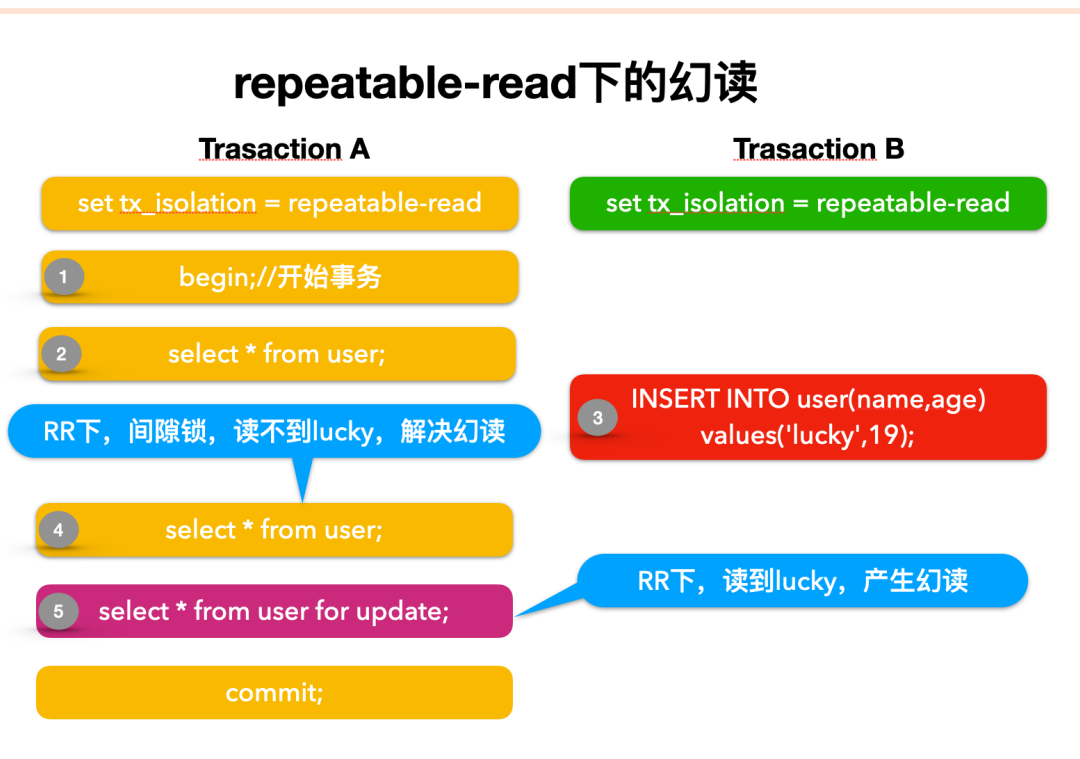
Trasaction A
Trasaction B
set tx_isolation = repeatable-read
set tx_isolation = repeatable-read
1 begin;//开始事务
2 select * from user where age>18;
RR下，A事务使用nextkey锁，B事务无法插入，解决幻读
3 INSERT INTO user(name,age) values('lucky',19);
4 select * from user;
commit;

- 间隙锁不能解决幻读的情况（对于当前读下的幻读）
  - **InnoDB中的REPEATABLE READ这种隔离级别通过间隙锁+MVCC解决了大部分的幻读问题，而对于当前读（SELECT FOR UPDATE、UPDATE、DELETE等操作）还是会产生幻读的现象的。**如果想要彻底解决幻读的问题，在InnoDB中只能使用Serializable这种隔离级别
  - 例



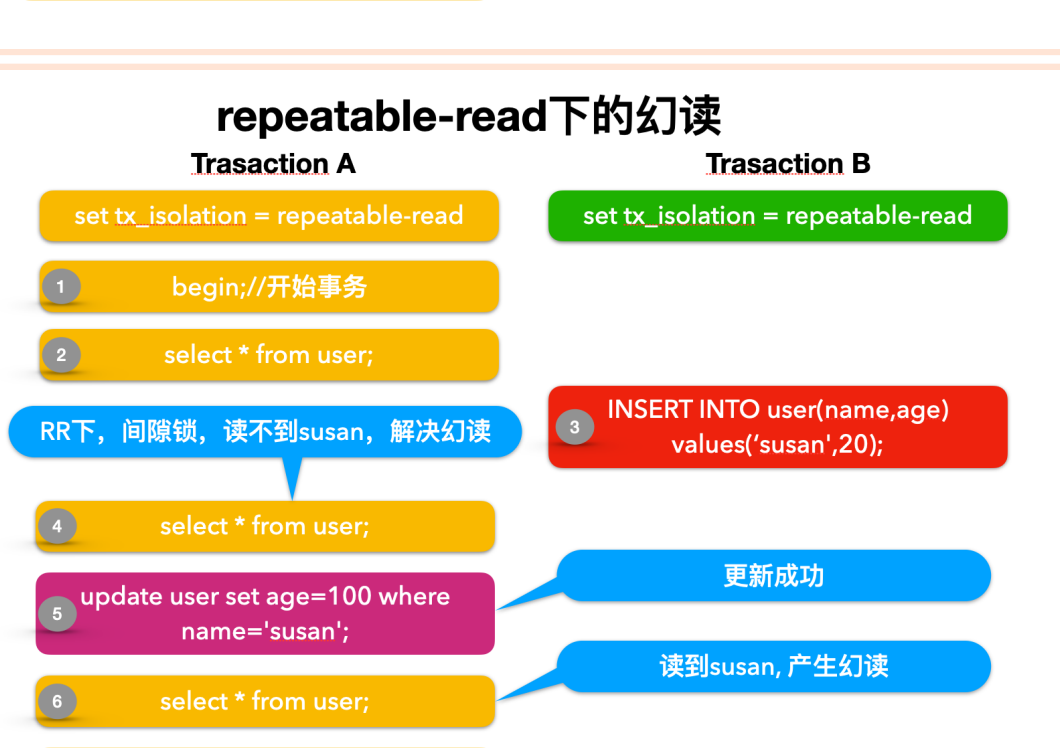
repeatable-read下的幻读

Trasaction A
Trasaction B
set tx_isolation = repeatable-read
set tx_isolation = repeatable-read
1 begin;//开始事务
2 select * from user;
3 INSERT INTO user(name,age) values('lucky',19);
如果没有第2步，则会幻读。
不是事务开始时产生readview，而是第一次select时产生readview，此时仍能读到lucky
4 select * from user;
commit;



repeatable-read下的幻读

Trasaction A
Trasaction B
set tx_isolation = repeatable-read
set tx_isolation = repeatable-read
1 begin;//开始事务
2 select * from user;
3 INSERT INTO user(name,age) values('lucky',19);
RR下，间隙锁，读不到lucky，解决幻读
4 select * from user;
5 select * from user for update;
RR下，读到lucky，产生幻读
commit;



repeatable-read下的幻读

Trasaction A
Trasaction B
set tx_isolation = repeatable-read
set tx_isolation = repeatable-read
1 begin;//开始事务
2 select * from user;
3 INSERT INTO user(name,age) values('susan',20);
RR下，间隙锁，读不到susan，解决幻读
4 select * from user;
5 update user set age=100 where name='susan';
更新成功
6 select * from user;
读到susan, 产生幻读
commit;

Mysql锁

- 间隙锁
  - 应用
    - 1. 对主键或唯一索引，如果当前读时，where条件全部精确命中(=或者in)，Innodb存储引擎会对next-key lock进行优化，将其降为record lock（记录锁），即仅锁住索引本身，而不是范围。
    - 2. 没有索引的列，当前读操作时，会加全表gap间隙锁，生产环境要注意。
    - 3. 非唯一索引列，如果where条件部分命中(>、<、like等)或者全未命中，则会加附近Gap间隙锁。
  - RR下，对于快照读来说，幻读的解决是依赖mvcc解决。而对于当前读则依赖于gap-lock解决。
  - 间隙锁只有在 Read Repeatable 、Serializable 隔离级别才有
- 死锁解决
  - - 查出的线程杀死 kill SELECT trx_MySQL_thread_id FROM information_schema.INNODB_TRX;
  - - 设置锁的超时时间:Innodb 行锁的等待时间，单位秒。可在会话级别设置，RDS 实例该参数的默认值为 50（秒）。生产环境不推荐使用过大的 innodb_lock_wait_timeout参数值该参数支持在会话级别修改，方便应用在会话级别单独设置某些特殊操作的行锁等待超时时间，如下：set innodb_lock_wait_timeout=1000; —设置当前会话 Innodb 行锁等待超时时间，单位秒。

mysql应用

- MySQL的limit工作原理 — 先读取前面n条记录，然后抛弃前n条，读后面m条想要的，所以n越大，偏移量越大，性能就越差
- 分页
  - 传统分页查询 — SELECT c1,c2,cn... FROM table LIMIT n,m
  - MySQL分页查询的性能优化
    - 尽量给出查询的大致范围 — SELECT c1,c2,cn... FROM table WHERE id>=20000 LIMIT 10;
    - 子查询法 — SELECT c1,c2,cn... FROM table WHERE id>= (SELECT id FROM table LIMIT 20000,1) LIMIT 10;
    - 高性能MySQL一书中提到的只读索引方法
      - 优化前SQL — SELECT c1,c2,cn... FROM member ORDER BY last_active LIMIT 50,5
      - 优化后SQL — SELECT c1, c2, cn .. . FROM member INNER JOIN (SELECT member_id FROM member ORDER BY last_active LIMIT 50, 5) USING (member_id) — 优化前的SQL需要更多I/O浪费，因为先读索引，再读数据，然后抛弃无需的行。而优化后的SQL(子查询那条)只读索引(Cover index)就可以了，然后通过member_id读取需要的列
      - 第一步用用程序读取出ID，然后再用IN方法读取所需记录
        - 优化前SQL — 优化后SQL
        - 优化后SQL — SELECT c1, c2, cn .. . FROM table WHERE id IN (id1, id2, idn.. .)
- jdbc-sharding分页
  - 问题 — sharding-jdbc的做法就改写我们的sql, 先查出来所有的数据, 再做归并排序
    - 查询第2页时

```
原sql是:
select * from ORDER_00 where create_tm >= ?  and create_tm <= ? limit 10 ,10 ;
select * from ORDER_01 where create_tm >= ?  and create_tm <= ? limit 10 ,10 ;
select * from ORDER_02 where create_tm >= ?  and create_tm <= ? limit 10 ,10 ;
会被改写成:
select * from ORDER_00 where create_tm >= ?  and create_tm <= ? limit 0 ,20 ;
select * from ORDER_01 where create_tm >= ?  and create_tm <= ? limit 0 ,20 ;
select * from ORDER_02 where create_tm >= ?  and create_tm <= ? limit 0 ,20 ;
```

    - 查询第3页时

```
原sql是:
select * from ORDER_00 where create_tm >= ?  and create_tm <= ? limit 20 ,10 ;
select * from ORDER_01 where create_tm >= ?  and create_tm <= ? limit 20 ,10 ;
select * from ORDER_02 where create_tm >= ?  and create_tm <= ? limit 20 ,10 ;
会被改写成:
select * from ORDER_00 where create_tm >= ?  and create_tm <= ? limit 0 ,30 ;
select * from ORDER_01 where create_tm >= ?  and create_tm <= ? limit 0 ,30 ;
select * from ORDER_02 where create_tm >= ?  and create_tm <= ? limit 0 ,30 ;
```

  - 解决方案
    - scroll就是滚动
      1. 查询第1页
         select * from order limit 0, pageSize;
      2. 记录第1页的最大id: maxId
      3. 查询第2页
         select * from order where id > maxId limit pageSize
      4. 把maxId更新为第2页的最大id
      .. 以此类推
    - 不分表(分区 tidb mongoDb ES)
    - 离线计算+有损服务 — 使用mysql+nosql结合的方式, 例如常规读写操作mysql, 分页查询走ES等等
- 分库分表
  - 分表逻辑
    - 如果1行大于100个字节，那么就是1000万一张表
      如果1行小于100个字节，那么就是5000万一张表
      siezOf(row)>100B, 1kw
      siezOf(row)<100B, 5kw
    - 如果只是主键查询，亿也可，主要看是业务场景吧
  - 分片规则
    - 数值取模
    - 数值范围 — 系统消息可以这样使用。如果是支持30天查询的话，可以跨库双写，如果是按年分的话，每个库多写一个月的数据
    - 映射表 — 如果2列都是查询条件，可以考虑建立关联映射表
    - 基因法
      - 生成uid时，分布式唯一ID生成方案，再加上最后3位bit值=f(login_name)。当查询login_name时，只需计算f(login_name)%8的值，就可以定位到具体的库
      - 什么是分库基因
        - 是什么 — 通过uid分库，假设分为16个库，采用uid%16的方式来进行数据库路由，这里的uid%16，其本质是uid的最后4个bit决定这行数据落在哪个库上，这4个bit，就是分库基因
        - 使用 — 通过uid分库，假设分为16个库，采用uid%16的方式来进行数据库路由，这里的uid%16，其本质是uid的最后4个bit决定这行数据落在哪个库上，这4个bit，就是分库基因 — 将分库基因加入到tid的最后4个bit，拼装成最终的64bit帖子tid — 保证了同一个用户发布的所有帖子的tid，都落在同一个库上，tid的最后4个bit都相同
          - 通过uid%16能够定位到库
          - 通过tid%16也能定位到库
  - 分片后的查询
    - 跨节点关联查询 join 问题
      - 字段冗余
      - 数据组装
      - ER分片，将那些存在关联关系的表记录存放在同一个分片上
    - 扩容 — 加从库，多节点进行主从同步，等完全一致后，进行主从分离，然后主和从形成新的集群，然后执行新的分片规则即可

配置优化

- InnoDB 参数
  - innodb_buffer_pool_size — 设置为可用内存的 70-80%，提高数据和索引的缓存能力
  - innodb_log_file_size — 增大日志文件大小，减少日志切换频率，提高写性能
  - innodb_flush_log_at_trx_commit — 设置为 2 或 0 以提高写性能，降低数据持久性要求时使用 — innodb_flush_log_at_trx_commit 设置为 2 时，事务提交时日志缓冲区的数据会写入日志文件，但不立即刷新到磁盘。InnoDB 每秒会将日志文件刷新到磁盘一次
- 查询缓存
  - query_cache_type — 根据工作负载适当开启或关闭查询缓存，避免不必要的开销
  - query_cache_size — 设置合理的查询缓存大小
- 连接和线程管理
  - max_connections — 根据实际需求调整最大连接数，避免连接过多导致的资源耗尽
  - thread_cache_size — 适当增加线程缓存，提高线程重用效率
- SQL 查询优化
  - 索引优化
  - 查询重写
- TPS优化
  - innodb_buffer_pool_size
    - 描述 — InnoDB 存储引擎用于缓存数据和索引的内存大小
    - 优化建议 — 增大这个值可以减少磁盘 I/O，提高查询速度。一般建议设置为物理内存的 70-80%
  - innodb_log_file_size
    - 描述 — 日志文件的大小
    - 优化建议 — 较大的日志文件可以减少写入频率，从而提高性能。通常建议设置为物理内存的 25%
  - innodb_flush_log_at_trx_commit
    - 描述 — 控制事务日志的刷新方式
    - 优化建议 — 设置为 2 可以在牺牲部分持久性（有可能丢失最后一秒的数据）的情况下显著提高 TPS
  - innodb_flush_method
    - 描述 — InnoDB 如何将数据刷到磁盘
    - 优化建议 — 设置为 O_DIRECT 可以减少操作系统缓存层的影响，直接写入磁盘，提高性能
  - innodb_thread_concurrency
    - 描述 — 限制 InnoDB 并发线程的数量
    - 优化建议 — 设置为 0（自动管理）或一个合理的值，通常在 8 到 16 之间，根据实际情况调整
  - innodb_io_capacity 和 innodb_io_capacity_max
    - 描述 — InnoDB 的 I/O 处理能力
    - 优化建议 — 根据磁盘的 IOPS 能力设置，通常可以从 1000 开始，逐步调整到接近磁盘的物理能力
  - query_cache_type 和 query_cache_size
    - 描述 — 查询缓存的相关设置
    - 优化建议 — MySQL 8.0 及以上版本已移除查询缓存。如果是旧版本，建议禁用查询缓存 (query_cache_type = 0) 来减少锁争用
  - max_connections
    - 描述 — 最大连接数
    - 优化建议 — 根据实际需要和服务器资源配置调整，不宜过高，以免超出资源承载能力
  - innodb_buffer_pool_instances
    - 描述 — InnoDB 缓冲池的分片数量
    - 优化建议 — 设置为 8 或 16 可以减少争用，增加并发
  - transaction-isolation
    - 描述 — 事务隔离级别
    - 优化建议 — 选择 READ COMMITTED 代替 REPEATABLE READ 来减少锁争用，但要根据具体业务需求选择合适的隔离级别

QA

- MySQL 的 NULL 值是怎么存放的
  - InnoDB 存储引擎提供的Compact格式

变长字段的长度列表	NULL 标识位	记录头信息	列 1 数据	列 2 数据	列 ... 数据

  - NULL占用空间 — 空值''的长度是0，是不占用空间的；而的NULL长度是NULL，是需要占用额外空间的，所以在一些开发规范中，建议将数据库字段设置为Not NULL,并且设置默认值''或0。
  - NULL对索引的影响
    - MySQL中某一列数据含有NULL，并不一定会造成索引失效
    - MySQL可以在含有NULL的列上使用索引
    - 在有NULL值得字段上使用常用的索引，如普通索引（NORMAL）、复合索引、全文索引（FULLTEXT）、唯一索引（UNIQUE）等不会使索引失效。但是在使用空间索引（SPATIAL）的情况下，该列就必须为 NOT NULL。
      - 在复合索引中，如果查询不使用索引的前导列（即索引定义中的第一个列），或者前导列经常是NULL，那么索引的其余部分也可能不被利用。这是因为MySQL通常无法有效地跳过复合索引中的前导NULL值来找到后续列的索引条目
      - 空间索引要求数据是具体的空间对象，不能是NULL
- 索引失效的操作(**函隐不等或前导，复限不均选择低**)
  1. 在索引列上使用函数或表达式 — 当在WHERE子句中对索引列使用函数或表达式时（例如SELECT * FROM table WHERE YEAR(column) = 2021）
  2. 隐式类型转换 — 如果查询条件中的数据类型与索引列的类型不匹配，MySQL可能需要对数据进行类型转换，这可能会导致无法使用索引。例如，在INT类型的索引列上使用字符串类型的比较（SELECT * FROM table WHERE int_column = '123'）
  3. 使用NOT EQUAL (<>)运算符 — 使用<>或!=运算符可能会导致MySQL扫描整个表，因为索引无法有效地排除与条件不匹配的行
  4. 使用OR条件连接不同列 — 如果在WHERE子句中使用OR来连接不同的列，而这些列分别有各自的索引，MySQL可能无法同时利用这些索引（除非使用索引合并优化） — 索引合并的优化技术，它可以在某些情况下被用来处理包含OR条件的查询。索引合并涉及到同时查阅多个索引并且合并结果
  5. 使用前导模糊搜索 — 使用LIKE操作符进行前导模糊搜索（例如SELECT * FROM table WHERE column LIKE '%value'）会导致索引失效
  6. 复合索引的列顺序不当
  7. 限制索引选择的SQL提示（IGNORE INDEX, FORCE INDEX） — 通过使用SQL提示来强制或忽略索引的使用可能会导致优化器无法选择最佳索引
  8. 数据分布不均匀 — 如果索引列的值分布非常不均匀，优化器可能判断使用全表扫描比使用索引更高效，尤其是当查询结果包含大量数据行时
  9. 选择性低的索引 — 如果索引的选择性（不同值的比例）很低，比如性别列（男/女），则这种索引可能不会被MySQL查询优化器使用